

AI Assignment-1

8
10

PAGE NO.

Uschawan
10/2/25

- 1] Explain Turing test designed for satisfactory operational definition of AI.
- 1) The Turing test provided by sir Alan Turing was designed to provide satisfactory operational definition of AI.
- 2) It is a method to determine machine's ability to exhibit intelligent behaviour.
- 3) In the test a human being judge interacts with a machine & a human without knowing which is which. If a judge cannot reliably distinguish the machine is considered to have passed the test, & demonstrate a level of artificial intelligence comparable to human.
- 4) During test human function as questioner while the second human & computer function as respondents.
- 5) After preset length of time the questioner is then asked to determine which was human & which was computer.
- 6) The test is repeated many times. If questioner makes correct determination half of test runs or less the computer is considered to have artificial intelligence.
- for now, we note that programming computer to pass a rigorously applied test provided plenty to work on.
- The computer would need to possess following capabilities.

Natural Language Processing

- NLP to enable successful communication in english

* knowledge representation

- To store what it hears or knows
Automated Reasoning

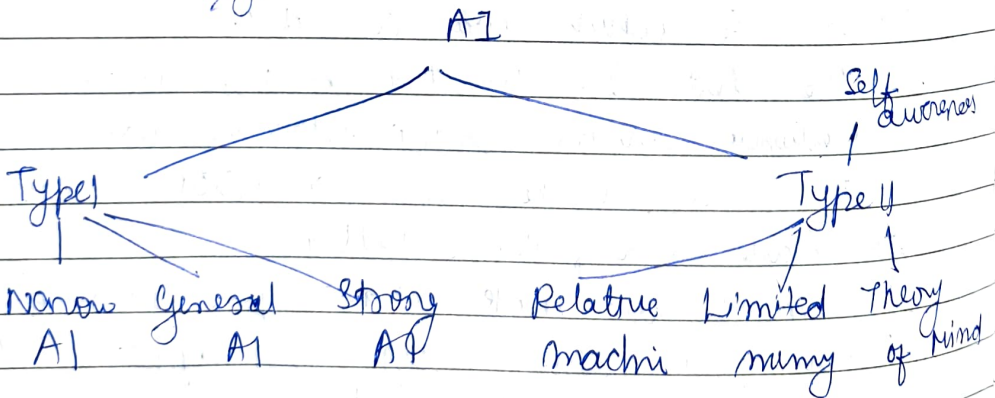
- To use stored information to answer question & draw new conclusions
Machine learning

- To adapt to new circumstances.

Explain different definition of AI?

① AI is a branch of computer science by which we can relate, create intelligent machine which can behave like humans, think like humans, & able to make decisions.

② Artificial intelligence can be divided into various type



① Weak AI :-

1) Narrow AI is a type of AI which is able to perform a dedicated task, with



intelligence. The most common & currently available AI is narrow AI.

- ii) Narrow AI cannot perform tasks beyond specific tasks.
- iii) Some examples are Chess AI, purchasing suggestions on e-commerce website.

2) General AI

- i) General AI is a type of AI which could perform any intellectual task with efficiency like human.
- ii) The idea behind general AI is to make a system which could be smarter & think like humans by its own.
- iii) Currently there is no such system which can perform task perfect as human.

3) Super AI

- i) Super AI is a level of AI of which machines could surpass human intelligence.
- ii) Super AI can perform tasks better than humans.
- iii) Super AI is hypothetical concept of AI.

Type 2: functionality -

① Reactive Machine

- purely reactive machines are most basic type of AI.
- These machines only focus on current scenarios & react on it as per possible best action.

2) Limited Memory :-

- Limited memory machines can store experience or same data for short period
- These machines can use stored data for limited period only
- Self driving cars are best example

3) Theory of Mind :-

- Theory of mind AI, should understand human emotions, people, beliefs $\rightarrow$ be able to interact socially like humans.
- These type of AI are still - not developed

4) Self-awareness :-

- Self aware AI is future of Artificial intelligence
- These machine will be super intelligent & will have their own consciousness

Q.3] Define rationality & rational agent. Give eg.
 $\rightarrow$ Rationality of an agent is measured by its performance. measure rationality can be judged based on following:

- performance measure which defines the success criteria
- Agents prior knowledge of its environment
- Best possible action that agent performs
- The sequence of percepts

Rational Agent :-

A rational agent is an agent which has clear preference, makes uncertainty & acts in

way to maximise its performance

- A rational agent always performs the right things the right action means action which causes AI to be most successful

- Rational agent is capable of taking best possible action in any scenario

ex: Automated taxi driver.

- 1) Performance Measure: Safe, fast, legal, comfortable
- 2) Environment: Roads, other traffic, customs
- 3) Actuators: steering, ~~steer~~ wheel, accelerator,
- 4) Sensors: camera, sonar, speedometer, GPS,

4) Describe different types of environment application of AI agents

→ ① An environment is everything in world which surrounds the agent, but it is not part of the agent itself.

② An environment can be described as a situation in which an agent is present

③ The environment is where agent lives, operates & provide agent with something to sense

Types:

① Fully observable vs partially observable

- If an agent sensor can sense or over the complete state of an environment at each point of time then it is fully observable environment, else it is partially observable

② Deterministic vs stochastic

- If an agent's current state & selected action

can completely determine the next state of environment, then such environment is called 'deterministic' environment.

- A stochastic environment is random in nature.

3) Episodic vs. Sequential

- If an episodic environment, there is series of one shot actions & only current percept is required.

- However, in sequential environment, an agent requires memory of past actions.

4) Single agent vs Multiagent

- If only single agent is involved in an environment & operating by itself such environment is called single agent environment.

- However if there are multiple agents it is called multi agent environment.

5) Static vs Dynamic

- If environment can change itself while an agent is deliberating then such environment is called dynamic environment, else it is a static environment.

6) Discrete vs Continuous

- If in an environment there are finite number of percepts & actions that can be performed within, then it is called discrete, else it is continuous environment.

7) Accessible vs Inaccessible.

- If an agent can obtain complete & accurate information about states then such environment is called accessible environment else it is called inaccessible environment.

5) Core PEAS description for robot soccer player.
 → PEAS is a type of model on which an AI agent works upon, when we define an AI agent then we can group its parameters under PEAS

- ① P - Performance measure
- ② E - Environment
- ③ A - Actuator
- ④ S - Sensors

Robot soccer player:

- ① Performance measure: Number of goals, speed, legel, ball, Control, make goals, win games
- ② Environment: Team players, opponent, ground, Audience, goal net
- ③ Actuators: Joint angles, motors, communication
- ④ Sensors: Camera, infrared sensors, vision sensors, proximity.

6) What is PEAS? Give PEAS for part parking robot?

→ PEAS is a type of model on which AI agent works upon.

P: Performance Measure

- Objective for success of task

E: Environment:

- It is the real environment where the agent needs to deliberate actions

A: Actuator.

- tools using which agent performs action in environment

S: Sensors.

- Sensors are tools which agent uses to sense environment.

Part Picking Robot:

Performance Agent:

Efficiency, no. of parts, accuracy, adaptability, environment.

storage area, container used, other robots, etc

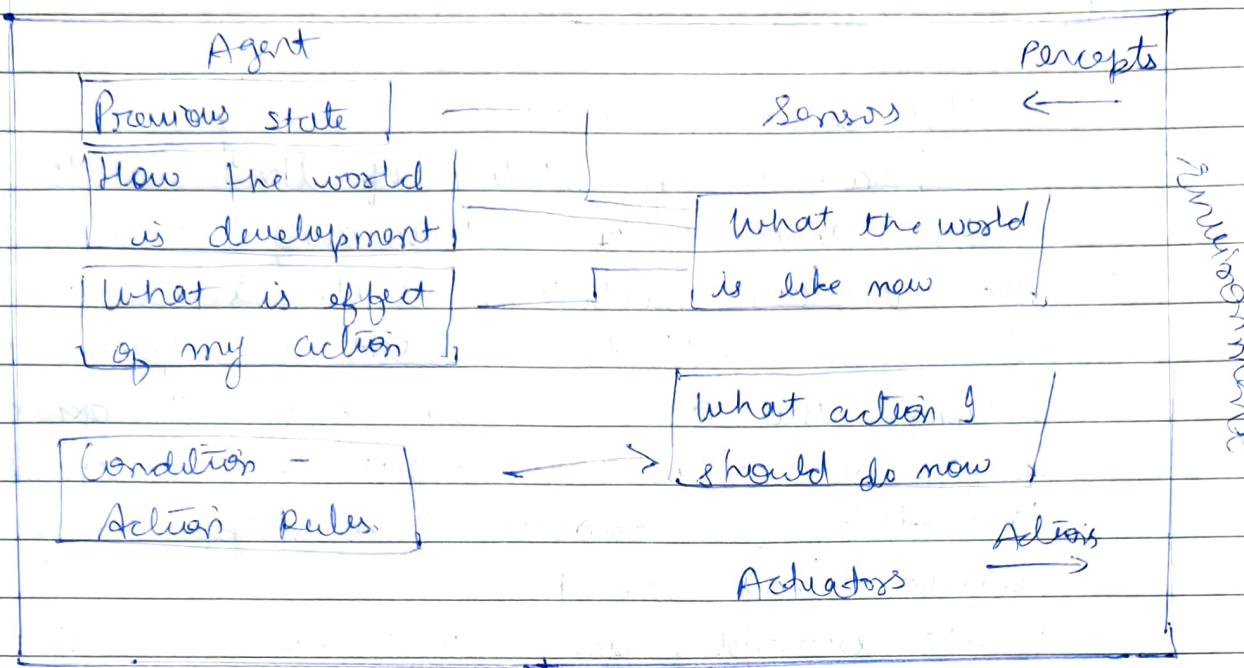
Actuators:

Robotic arm, mobility mechanism, arm with tool tips to pick & chop parts.

Sensors:

Camera, range sensors, joint angle sensors, proximity sensors.

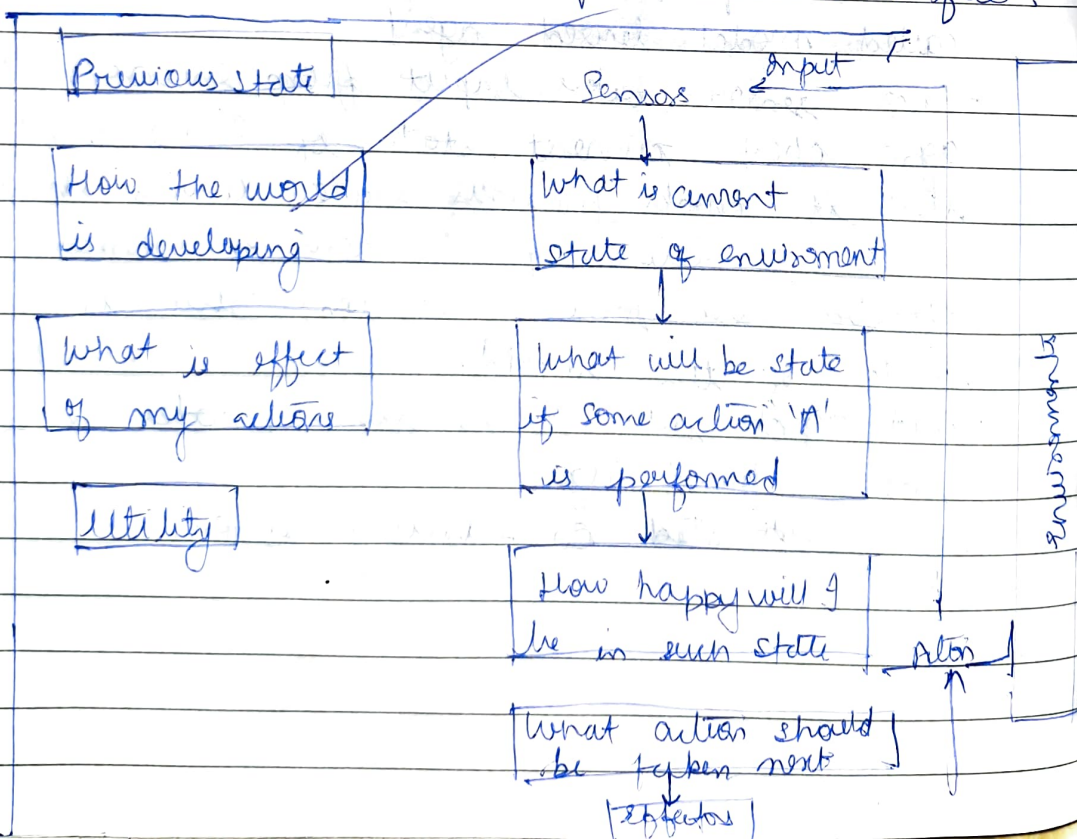
Q2] Explain model based reflex agent



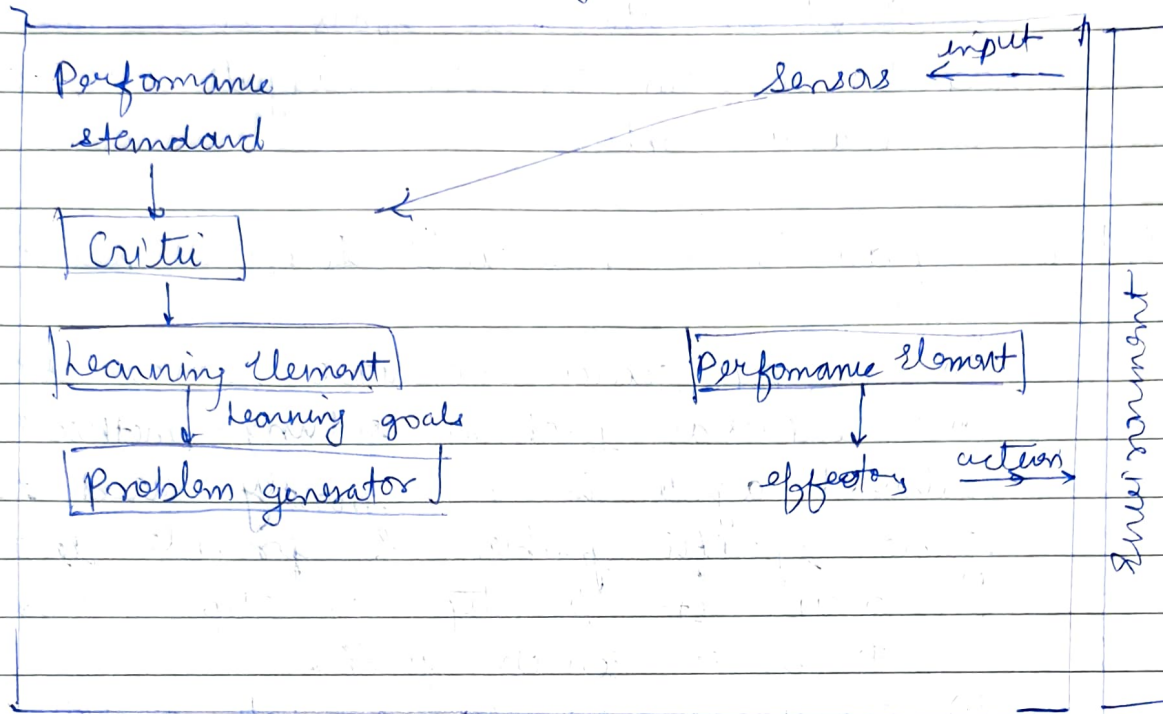
- Model based reflex agents handle partially observable environment which cannot be handled well by simple reflex.
- An agent which performs action based on current input & one previous input is called model based agent.
- One sensor takes input from environment, agent check current state of environment, after that it checks previous state which shows how world is developing & how environment was affected by previous action. This is called model of the world.
- And agent which uses such model is called model-based agent.
- eg. Automated car-driving system.

of utility based agents

- Utility function is used to map a state to measure of utility of that state. We define measure of determining how advantageous particular state is.
- The term utility is used to depict how happy the agent is to find out generalized performance measure, various world states according to exactly how happy they would make agent.
- Take example, you might have used google maps to find route with least possible time. Same logic is used in automatic car driving agent.
- Goals utility based automatic car agent can be used to reach location safely within least amount of time & save fuel.



Q9) Explain learning based agent.



- An learning based agent is an agent that can learn from its past experience.
- A learning based agent has four components
 - Critic :- It is the one who compares sensors input specifying effect of agents action on environment with the performance standard.
 - Learning Element: The component responsible to learn difference between performance standard & feedback from critic. It is supposed to understand expected behaviour & enhance the standard.
 - Performance Element:

Based on current percept received from sensors & input obtained by the learning element, performance element is responsible to choose the action to act upon the external environment.

• Problem generator,

Based on the new goals learnt by learning agent, problem generator suggests new or alternative actions.

Q.10] Explain how you will formulate search problem

→ In an AI search technique search techniques are universal problem solving methods. Rational agents in AI mostly used to solve specific problem & provide best results.

① Search: It is step by step procedure to solve search problem.

- i) Search space: It represents a set of possible solutions, which system may have.
- ii) Start state: It is a state from where agent begins search.
- iii) Goal test: It is function which observes the current state & returns whether goal state is achieved or not.
- iv) Search tree: A tree representation of search problem is called search tree.
- v) Actions: All actions available to agent.
- vi) Transition model: What each action can do.
- vii) Path cost: A function which assigns numerical value to each path.

Q.11] Explain steps in problem formulation.

- 1. Initial state: The initial state in which agent starts its



2. Actions: It is set of actions that can be executed or applicable in possible states. i.e. transition model.
3. Success function: It is function that returns a state on executing action on current state.
4. Goal test: It is a test to determine whether current state is defined goal state by comparing current state with defined goal state, called explicit goal test.
Whereas in some cases, states cannot be defined explicitly, but needs to generated by computations it is called implicit goal test.
5. Path cost: It is simply cost associated with each step taken to reach goal.
6. Problem solution: A solution to problem is sequence of actions chosen by problem solving agent that leads to goal state.

12] Formulate & puzzle problem

1	2	3
4	8	-
7	6	5

Initial state

1	2	3
4	5	6
7	8	

Goal state

Above is a typical puzzle problem

States:

Initial state: $\{ \{1, 2, 3\}, \{4, 8, 6\}, \{7, 6, 5\} \}$

Action: Block space can move left, right, up & down.

Goal state: $\{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 0\}$.

Solution:

$\{1, 2, 3\}, \{4, 8, 0\}, \{7, 6, 5\} \rightarrow \{1, 2, 3\}, \{4, 8, 5\}, \{7, 6, 0\}$
 $\{1, 2, 3\}, \{4, 8, 5\}, \{7, 0, 6\} \rightarrow \{1, 2, 3\}, \{4, 6, 5\}, \{7, 8, 0\}$
 $\{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 0\} \rightarrow \{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 0\}$

Path cost = 5 steps.

13] Compare uniform search.

Aspect	BFS	DFS	VCS	DLS	IDFS
Completeness	Complete	Not Complete	Complete	Depend on limit	Complete
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^{ \epsilon })$	$O(b^l)$	$O(b^m)$
Memory usage	High	Low	Moderate to high	Low to moderate	Optimal
Optimality	Optimal	Not Optimal	Optimal	Not always optimal	Optimal
Suitability	Small graphs	Large graphs	Path Cost problem	known depth	Unknown depth

14] Write short note on uniform search.

- ① Uniform search is class of general purpose search algorithm which operate in brute force way
- ② The term uniform means they only have information about start state & goal state along with problem definition.
- ③ These techniques can generate successor state & can distinguish a goal state.
- ④ All these search techniques are distinguished by the order in which nodes are expanded.
- ⑤ Also called as blind search
- ⑥ Types of uniform search algorithm.
 - a) Breadth First search
 - b) Depth First search
 - c) Depth Limited search
 - d) Iterative deepening depth first search
 - e) Uniform cost search
 - f) Bidirectional search

15] Compare between Informed & Uniform Search technique.

Informed search	Uniform search
1. It is known as heuristic search.	1. It is known as blind search.
2. It uses knowledge for searching process.	2. It doesn't use knowledge for searching process.

3. It is more quick

It is slower

4. Cost is low

Cost is high

5. It is ^{less} lengthy

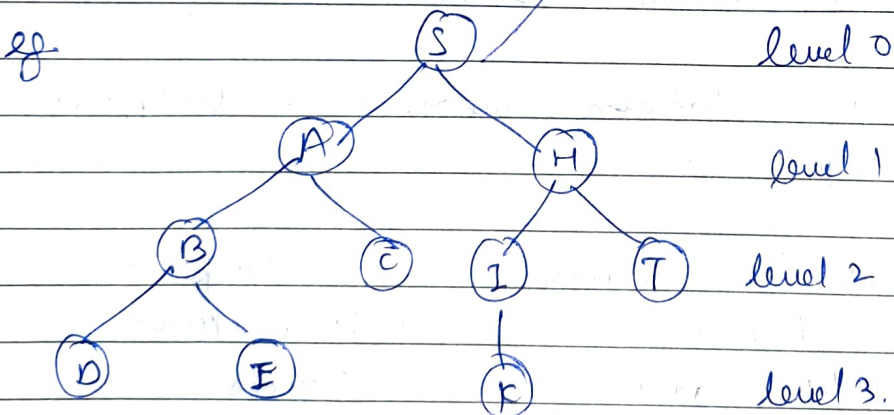
It is more lengthy

eg. A*,
greedy search.

eg. DFS
BFS.

16) Explain Depth First search

- 1) The depth first search or DFS explores or traverses data structures, such as tree & graphs.
- 2) It is a recursive algorithm for traversing tree or graph.
- 3) It starts from root node & follows each path to its greatest depth before moving to next path.
- 4) DFS uses stack.



Goal C.



It will start from S traverse to A, then D, then D & E

It will backtrack as E has no successor. After to C. Goal is found.

17] Explain BFS.

- ① Breadth First search is a search strategy that is used to traverse the tree.
- ② This algorithm searches breadth of tree or graph hence called as Breadth first search.

Advantages:

- ① BFS will provide solution if it exist.
- ② If there are more than one solution, then BFS will provide minimal solution.

Disadvantages:

- ① It requires a lot of memory, since each level of tree must be stored each level in memory.
- ② BFS needs a lot of time if solution is far away from root node.

18] Explain IDDFS.

- ① IDDFS stand for iterative deepening depth first search.
- ② It is combination of BFS or DFS.
- ③ It finds out the best depth in limit & does it by gradually increasing the limit until goal is found.
- ④ This algorithm performs depth first search upto certain depth limit.

⑤ It keeps increasing depth limit after each iteration

⑥ The IDDFS combines benefits of BFS & DFS search

Advantage:

- It combines benefits of BFS & DFS search algorithm.
- Memory efficiency

Disadvantage:

- Main Drawback is that it repeats all the work of previous steps.

19]

→ write a short note on Informed search algorithm

- Informed search techniques is further extension of basic uninformed search
- Main idea is to create additional information about search state space using knowledge of problem Domain so that search becomes more intelligent & efficient
- The evaluation function is developed for each state, which quantifies the desirability of expanding that state.
- All strategies use this evaluation function in order to select the next state under consideration, hence the name informed search
- The techniques are very efficient with time & space requirements.

ex. 1) A* Algorithm

2) IDA*

3) Best first search



- 20] Explain Heuristic function with example
- ① A heuristic function is a function that determines how near a state is to desired state
- ② Heuristic functions vary depending on problem & must be tailored to match the problem.
- ③ Heuristic is an a method of problem-solving where goal is to comeup with feasible solution in feasible amount of time.

Properties :

a) Admissible condition:-

If an algorithm produces an optimum solution it is considered Admissible.

b) Completeness:-

If an algorithm ends with solution it is considered complete

Direct Heuristic search Technique:-

Direct heuristic search technique may also called blind control strategy blind search or uniformed search they utilize on arbitrary sequence of operation

Weak Heuristic Technique:-

They are known as heuristic common strategy ex: informed search

For

Travelling Salesman Problem

- This problem can be brute forced for small number of cities
- The issue is well solved by nearest neighbour heuristic, which direct computer to always choose the closest unexplored city as next stop on path while this only sometimes offer optimal solution. It is frequently near enough that variation is insignificant.

Q21] Find Heuristic value for a particular state of the blocks world problem.

→ Block state problem.

1. Block world is the classic toy-world Problem of AI.
2. It has been used to develop AI system for vision, learning, language understanding & planning.
3. It consists of set of blocks placed on the table top.
4. The task is usually to stack the blocks in some predefined order.

example:-

Consider the example of Block world shown Below:

	D	
	B	
1	C	2
	A	

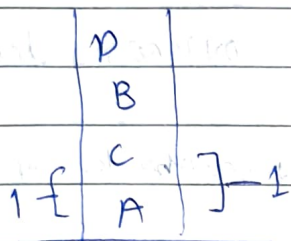
Initial state $H(S_1) = 2$

	D	
1	B	2
1	C	2
	A	

Goal
 $H(G) = 2$

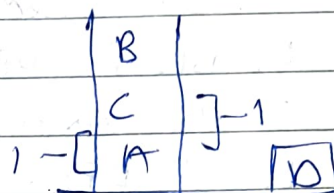


Heuristic value: Add one point for every block which is resting on the thing that it must be resting on. Therefore, heuristic value for initial state $H(S_1) = 2$ & for state 2 $H(S_2) = 2$.



Initial state

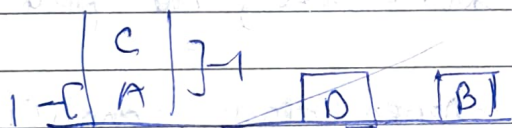
$$H(S_1) = 2$$



state 2

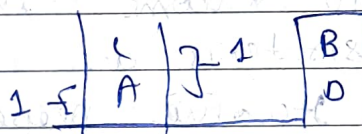
$$H(S_2) = 2$$

Now for state 2 there are 3 stage possible as shown below



Initial state

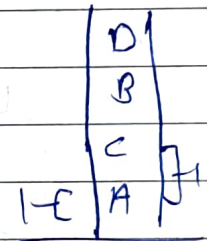
$$H(S_{3a}) = 2$$



Initial state

$$H(S_{3b}) = 2$$

state 3

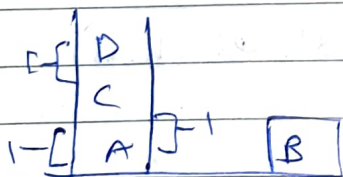


Initial state

state 2

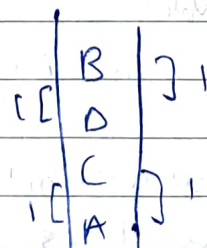
$$H(S_2)$$

In order to reach goal state we select S_{3a} therefore, heuristic value of $H(S_{3a}) = 2$.



State 4

$$H(S_4) = 3$$



Goal state

$$H(G) = 4$$

Now, Heuristic value for state 4 ($H(s_4)$) = 3 & from state 4 after one more the goal state is achieved

Hence Heuristic value for goal $H(G) = 4$.

Q22] Explain search strategy to overcome drawback of BFS & DFS.

→ Search strategies to overcome Drawbacks of BFS & DFS.

Both DFS & BFS has limitation:

- BFS: High memory as it stores all nodes at a level.
- DFS: Can get stuck in deep or infinite paths, making it inefficient.

To overcome these drawbacks, the following strategies are used:

1. Iterative Deepening Depth First Search (IDDFS): fixes DFS' infinite depth issue while using less memory than BFS.

- Uses depth-limited DFS, gradually increasing the depth limit.
- Ensures completeness like BFS but has memory efficiency of DFS.
- Best for problems where depth is unknown.

example:
Used in AI for game trees (eg chess)

2. Bidirectional search:

fixes BFS's high memory usage & speeds up search.

- Runs two BFS searches.



One from the start node $\rightarrow$ goal

One from the goal $\rightarrow$ start

- The search stops when both meet in middle
- Reduces search space from $O(b^d)$ to $O(b^{(d/2)})$ where b is branching factor & d is depth

example:

Used in shortest path algorithm (Google maps, ^{CRS})

B. A* Search Algorithm

Combines BFS completeness with DFS efficiency using heuristics.

- Uses $f(n) = g(n) + h(n)$, where:

$g(n)$ = cost from start to node

$h(n)$ = estimated cost to goal (heuristic)

- Guarantees an optimal path when using an admissible heuristic

- Example: Pathfinding in AI, robotics, & navigation

4. Greedy Best first Search

Uses heuristic to improve DFS but may not find the optimal solution

- Prioritizes nodes closest to the goal using $h(n)$.
- faster but can get stuck in local optimal
- e.g. example: used in AI for quick decision-making.

Q23] Explain A* Algorithm. What is the drawback of A*? Also show that A* is optimally efficient.

A* Algorithm:

The A* (A-star) algorithm is widely used

pathfinding & graph traversal algorithm. It finds the shortest path from a start node to a goal node using a combination of:

- $g(n)$: The cost to reach node n from the start node.
- $h(n)$: The estimated cost (heuristic) from node n to the goal.
- $f(n) = g(n) + h(n)$: The total estimated cost of the path passing through node n .

The algorithm prioritizes nodes with the lowest $f(n)$ value, balancing actual cost ($g(n)$) & estimated future cost ($h(n)$), ensuring efficient pathfinding.

Steps of A* Algorithm:

1. Initialize the open set with the start node.
2. Loop until the open set is empty or the goal is found:
 - Select the node with the lowest $f(n)$.
 - Expand the node & update its neighbours.
 - If a better path to a neighbour is found, update its $g(n)$ & recompute $f(n)$.
 - Add unvisited neighbours to the open set.
3. Terminate when the goal is reached or no path exists.

Drawbacks of A* Algorithms:-

1. High Memory usage

- A* stores all visited nodes, leading to excessive memory consumption in large graphs.



2. Performance Depends on heuristic

- If the heuristic function $h(n)$ is not well designed
- Overestimation (inadmissible heuristic) may cause the algorithm to slower

3. Computationally expensive

- The priority operator & heuristic calculations make it slow for complex maps.

A* is optimally efficient:

A* is optimally efficient when using an admissible & consistent heuristic. This means:

- Admissible heuristic: $h(n)$ never overestimates the actual cost
- Consistent heuristic: $h(n)$ satisfies the triangle inequality

Q.24) What do you mean by admissible heuristic function? Explain with example.

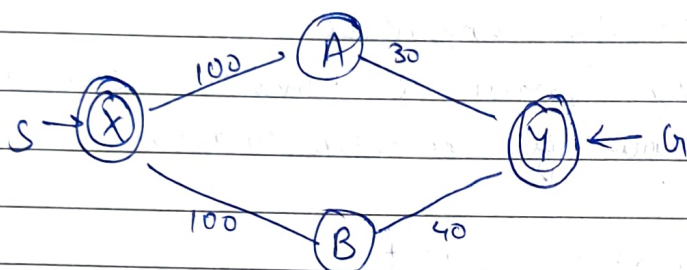
An admissible heuristic function is a heuristic function used in search algorithm (especially in A* search) that never overestimates the cost of reaching the goal. In other words, a admissible heuristic is always optimistic, meaning it either exactly estimates the cost or underestimates it.

A heuristic function $h(n)$ is admissible if for every node n , it satisfies:

$$h(n) \leq h^*(n)$$

- $h(n)$ is heuristic estimate of cost from node n to goal.
- $h^*(n)$ is true cost from n to goal.

eg.]



Case-1 :- Overestimation $h(n) \geq h^*(n)$

Let's suppose,

$$H(A) = 60, \quad H(B) = 50$$

$$F(n) = G(n) + h(n)$$

$$\therefore F(A) = 100 + 60 = 160$$

$$F(B) = 100 + 50 = 150$$

by comparing, $F(A) > F(B)$

So, choose path B & apply A*

$$F(Y) = G(Y) + h(Y)$$

$$= 140 + 0$$

$$= 140$$

[$h(Y) = 0$, goal state]

Case 2: Underestimation

Let's suppose

$$H(A) = 20, \quad H(B) = 10$$

$$F(n) = G(n) + h(n)$$

$$\therefore F(A) = 100 + 20 = 120$$

$$F(B) = 100 + 10 = 110$$

By comparing, $F(A) > F(B)$

So choose path & apply A^*

$$F(Y) = g(Y) + h(Y) \quad [h(Y) = 0, \text{ good state}]$$

$$= 140 + 0$$

$$= 140$$

As, $F(Y)$ is same in both circumstances, but in 2nd case by comparing the $F(A)$ with $F(Y)$ i.e. $120 < 140$ as it means we can go from node A $\therefore A^*$ will go the $F(A)$, which has value of 120.

$$F(Y) = g(Y) + h(Y)$$

$$= 130 + 0$$

$$= \underline{130}$$

$\therefore$ Based on all these calculations, we can say that this is the optimal value

Q25 Prove that A^* is admissible if it uses monotone heuristic

→ 1. Admissibility:- A^* is admissible if it always finds an optimal (least-cost) path to the goal when one exists

2. Monotone heuristic (consistent heuristic):

A heuristic $h(n)$ is monotone if for every node n & its successor n' , the following condition holds +

$$h(n) \leq c(n, n') + h(n')$$

where, $c(n, n')$ is the cost of the edge from n to n' . Also $h(\text{goal}) = 0$

This implies the A^* evaluation function

$$F(n) = g(n) + h(n)$$

is non-decreasing along any path i.e. $f(n) \leq F(n')$

2. Node Expansion Order.

Since A^* expands nodes with the smallest F value, it won't skip a cheaper path for a costlier one.

3. Optimality.

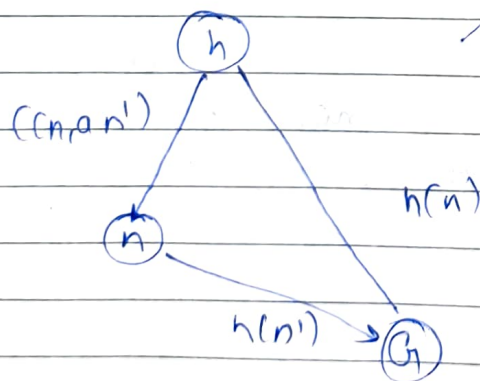
If A^* finds a goal node h with cost $g(h)$, any cheaper goal h' (if existed) would have been expanded first because:

$$F(h') = g(h') < F(h) = g(h)$$

This ensures A^* always finds the optimal path.

Since a monotone heuristic ensures non-decreasing F -values along paths, A^* cannot overlook a cheaper path once a more expensive one is expanded. This guarantees that A^* is admissible when using a monotone heuristic.

$$h(n) \leq c(n, a, n') + h(n')$$





Q.26] Compare following informed search algorithms based on performance measure with justification optional, time complexity & space complexity.

Criteria	Greedy BFS	A^*	Recursive BFS.
1) Optimality	Not optimal (can miss the best path)	Optimal (with admissible, consistent, heuristic)	Optimal (with heuristic, consistent heuristic)
2) Time complexity	$O(b^m)$ (in worst case)	$O(b^d)$ (exponential in depth d)	$O(b^d)$ (similar to A^* , but more efficient in prac.)
3) Space complexity	$O(b^m)$ (stores all nodes in memory)	$O(b^d)$ (stores all explored nodes)	$O(b^d)$ (linear least uses less mem.) by so
4) Performance justification	fast but can get stuck in local minima, focus only on $b(n)$	Balances cost so far g(n) & estimated cost $h(n)$; finds optimal path	similar to A^* least uses less memory by securing space with recursion

b = branching factor (avg no of child nodes per node)
 m = max. depth of search tree.
 d = depth & optimal solution.

Q 27]

Describe IDA* search algorithm giving suitable example

-
- ① Iterative Deeping A* (IDA*) is a search algorithm that combines the benefits of A* & Iterative deeping depth-first search (DFS)
 - ② It searches the state space using iterative deeping, where each iteration performs, a depth first search (DFS), with a cost threshold.
 - ③ The cost threshold is initially set to the heuristic estimate of the start node & increases progressively.

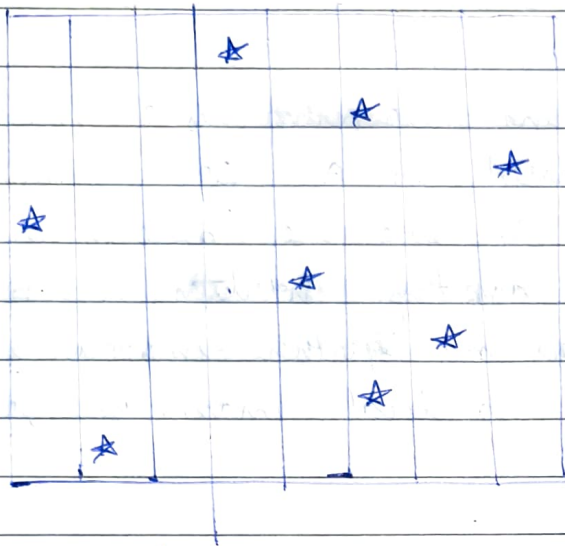
example →

8 Queens problem

The 8 - Queens problem is a classic example of the n - Queen problem where n queens have been placed on $n \times n$ chess board such that no two queens attack each other.

The state space of the problem can be represented as a tree where each node represent a configuration of the chessboard & each edge represents the placement of a queen. IDA* can be used to find the minimum no. of queens that need to be moved to reach the goal state.

IDA* is used to find the optimal solution by using a combination of depth-first search.



and a heuristic function to limit the search space. This algorithm incrementally increases the depth bound, allowing it to find the solution without exploring the entire state space which would be infeasible for larger problems.

Advantages:-

1. Optimal pathfindings.
2. Memory efficient: less limited memory compared to A^* .
3. Admissible heuristic
4. Efficient with large state space: handles large graphs efficiently.

Disadvantages:-

1. Repeated Node Exploration: does not store visited nodes.
2. Slower than A^* due to repeated exploration.
3. Higher computational cost.

Q28] Explain hill climbing algorithm with example. What are its limitations?

→ The Hill Climbing algorithm is a heuristic search algorithm used in AI for optimization problems. It is an iterative algorithm that starts with an arbitrary solution & makes incremental changes no further improvements can be made, reaching a local maximum or minimum.

Steps of Hill Climbing algorithm:

1. Start with an initial solution.
2. Evaluate the neighbors of current solution.
3. Move to the neighbor with the best evaluation.
4. Repeat the process until no better neighbor can be found.

example

If you want to find a mall from your current location. There are no possible paths with different directions to reach to the mall. The heuristic function will just give you the distance of each path which is reaching to the mall, so that it becomes very simple & time efficient for you to reach the mall.

Limitations:-

- Local maxima:- The algorithm may get stuck at a peak that is not the global maximum.



- Plateau Problem - If the landscape, has flat areas the algorithm might ender without progress.
- Ridges: when the optimal solution lies along a narrow ridge, the algorithm may struggle to navigate it
- No backtracking: Hill climbing does not remember past state, making it unable to escape suboptimal solutions.

8.29] What are the problems / frustrations that occurs in hill climbing technique? Illustrate with an example.

→ 1. Local Maxima:

The algorithm can get stuck at a peak that is not the highest one globally, preventing it from finding the best possible solution.

Example - Imagine a total navigating a hilly terrain. If it reaches a hill that appears to be the highest but is not the tallest one in the entire landscape, it stops prematurely.

2. Plateaus:

In a plateau, the algorithm may not find a direction of improvement because all neighbouring points have the same value, leading to random movements.

example:- A chess AI evaluating positions may struggle to make progress if multiple moves seem equally good, causing stagnation.

3. Ridge:- If the optimal path lies along a ridge, the algorithm might move in the wrong direction as it cannot follow a complete path effectively.

For example: A mountain climber trying to reach the peak but facing steep drops on both sides, making navigation difficult.

4. Stochastic Behaviour:- If there are multiple equally good choices, the algorithm may make arbitrary decisions, leading to inefficient results. Scenarios, multiple configurations might yield similar performance, causing inconsistency in decision-making.

Q.30] Write short note on genetic algorithm.

→ Genetic Algorithm (GAs) are a type of optimization technique inspired by the process of natural selection algorithms & are widely used for solving complex optimization & search problems. GAs operate on a population of potential solutions, evolving them over several generations to improve their fitness.



key steps in genetic Algorithm

1. Initialization: A population of potential solutions is randomly generated.
2. fitness evaluation: Each individual is assessed using a fitness function to determine its ^{quality} ~~val~~.
3. Selection: The fittest individuals are chosen for reproduction using methods like roulette wheel or tournament selection.
4. Crossover (recombination): Selected individuals exchange genetic material to create new offspring, improving diversity.
5. Mutation: Small random changes are introduced to maintain diversity & avoid premature convergence.
6. Replacement: The new offspring replace weaker individuals in the population.
7. Termination: The process continues until a stopping condition, such as a maximum number of generations or a satisfactory solution is met.

Genetic Algorithms are widely used in optimization, scheduling, machine learning, robotics, & other complex problem-solving domains.

Q.31] Explain how genetic algorithm can be used to solve a problem by taking a suitable example.

Genetic algorithms are used to solve optimization problems by simulating natural selection. Example:-

Solving a travelling salesman problem (TSP). The travelling salesman problem (TSP) involves finding the shortest route for a salesman to visit all cities once & return to the starting point.

Steps to solve using GA.

1. Initialization:- Generate a random population of possible city visit sequences.
2. fitness evaluation: Evaluate each route's total distance (shortest route are better).
3. Selection: Choose the best-performing routes using selection techniques like tournament selection or roulette wheel selection.
4. Cross over: Perform crossover on selected routes to create new offspring routes to create new offspring routes.
5. Mutation: Introduces small changes (like swapping cities) in a few routes to maintain diversity.



6. Replacement: The new generation replaces the old, keeping the best solutions.

7. Termination: The process repeats until an optimal or near-optimal solution is found.

Example: Computation:

a. Parent Chromosome

Chromosome A	1	5	3	2	6	4	7	9	8
Chromosome B	8	5	6	7	2	3	1	4	9

Child chromosomes offer arithmetic crossover i.e. adding bits of both chromosomes.

b. Child chromosomes.

Chromosome c	9	0	9	9	8	7	8	3	7
--------------	---	---	---	---	---	---	---	---	---

After applying value encoding mutation i.e. adding or subtracting 1 to 3rd & 4th bit

c. Child chromosome

Chromosome c	9	0	8	8	8	7	8	3	7
--------------	---	---	---	---	---	---	---	---	---

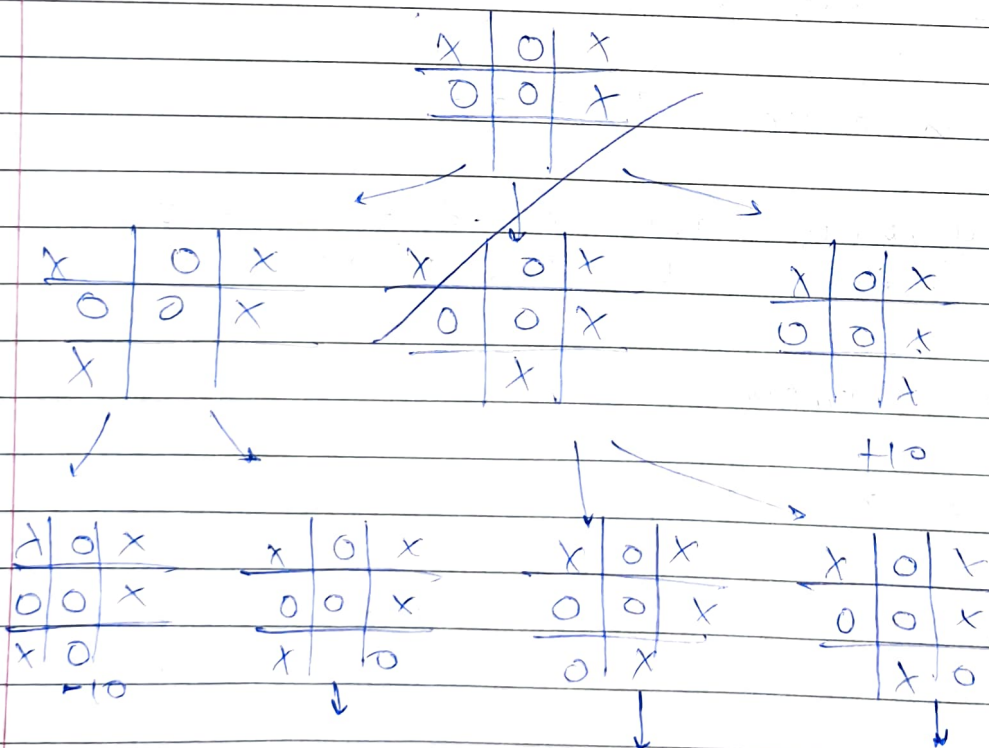
It can be observed that the child produced is much better than both parents

Q.32] Draw game tree of Tic-Tac-Toe problem.

The game tree of Tic-Tac-Toe represents of all possible moves from the initial state to terminal states (win, lose, or draw). Due to large number of possible board configuration, drawing the entire tree is impractical. However I can illustrate a partial game tree starting from an empty board.

Structure of Tic Tac Toe Game tree.

1. Root Node - Represents the empty board.
2. Branches - Each branch represents a valid move.
3. Levels - Alternating turns between X & O.
4. Leaf Nodes - Terminal states where someone wins or the game ends in a draw.



x	0	x
0	0	x
x	x	0

+0

x	0	x
0	0	x
0	x	x

+10

x	0	x
0	0	x
x	x	0

+0